

CIS 371 Web Application Programming

Cloud DataBase

Firebase Cloud Firestore



GRAND VALLEY
STATE UNIVERSITY®

Lecturer: **Dr. Yong Zhuang**

Exercise from the previous class

Practice: Responsive Product Card Dashboard

Requirements

- Create v-cols for all products defined in the script.
- Each v-col should contain a v-card showing:
 - name, price, and a Buy button
- Use Vuetify's grid system to make the layout responsive:
 - 4 columns per row on large screens
 - 2 columns per row on medium screens
 - 1 column per row on small screens



[Answer](#)

Why Cloud Data Stores?

- Highly scalable
- Usually built using No SQL technology
- Accessible to both web and mobile clients

No SQL = No DB Schema

SQL

vs.

noSQL

- Relational model
- Schema: relationship between tables and fields
- Popular examples
 - Oracle
 - DB2
 - MySQL
 - PostgreSQL

- Non-relational
- Schemaless Datastore
- Cloud Computing and Cloud Storage
- Rapid Development
- Popular examples
 - MongoDB
 - CouchDB
 - BigTable
 - Firebase Realtime DB
 - Firebase Cloud Firestore

Schema or Schemaless?

First	Last	G#	Major
Alice	Smith	12345678	Statistics
Brad	Jordan	23456789	History

Must redefine the SCHEMA to add a new column.

First	Last	G#	Major
Alice	Smith	12345678	Statistics
Brad	Jordan	23456789	History
Gary	deGroot	72551834	Biology
Ann	Hunt	78921631	Physics
Fay	Ross	72631235	English

Color	SocMedia	?
Green		
	IG, FB	
	TW	
Blue		
	LinkedIn	

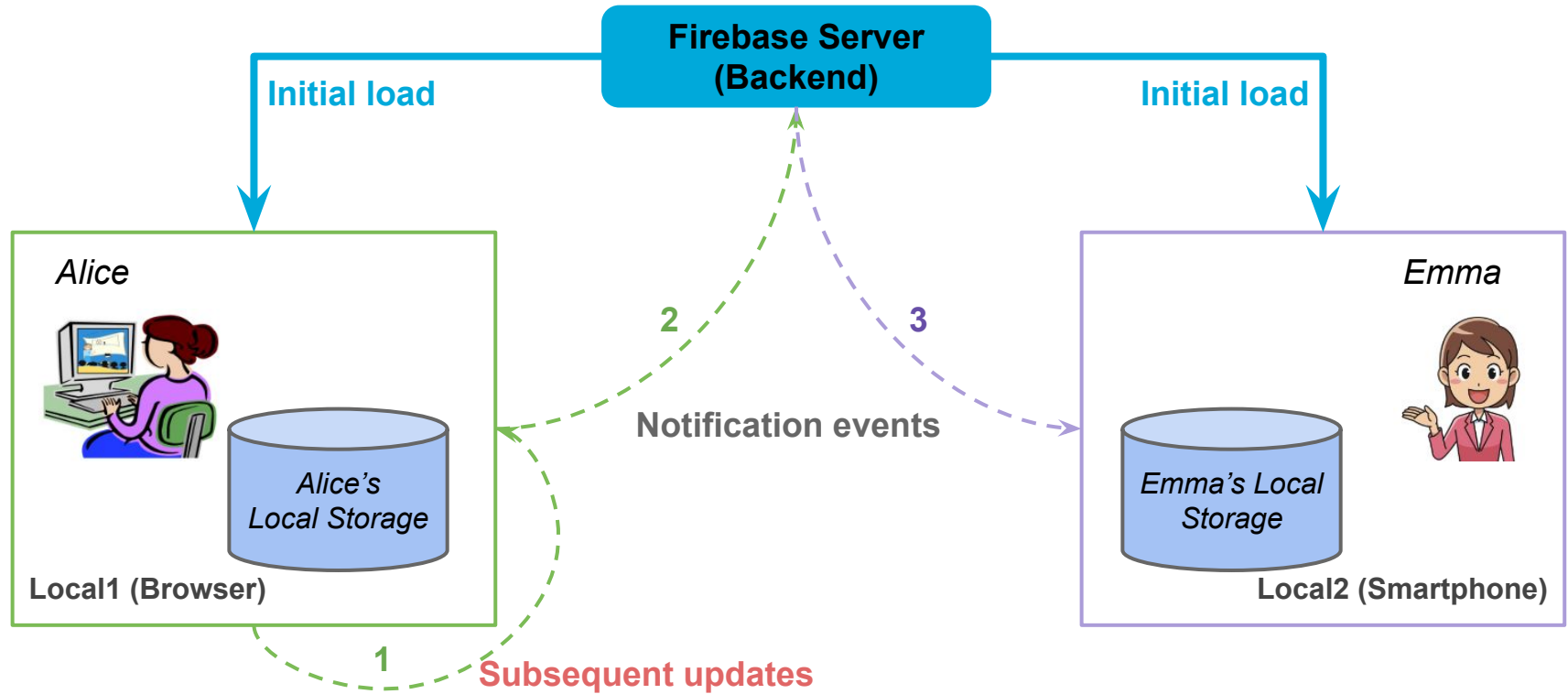
Firebase

- A collection of many products
- Cloud Firestore (beta since 2017, GA since 2019)
- Authentication
- Cloud Storage
- Realtime DB (beta since 2012, GA since 2014?)
- Cloud Messaging
- ML Kit
- Cloud Functions

Firebase

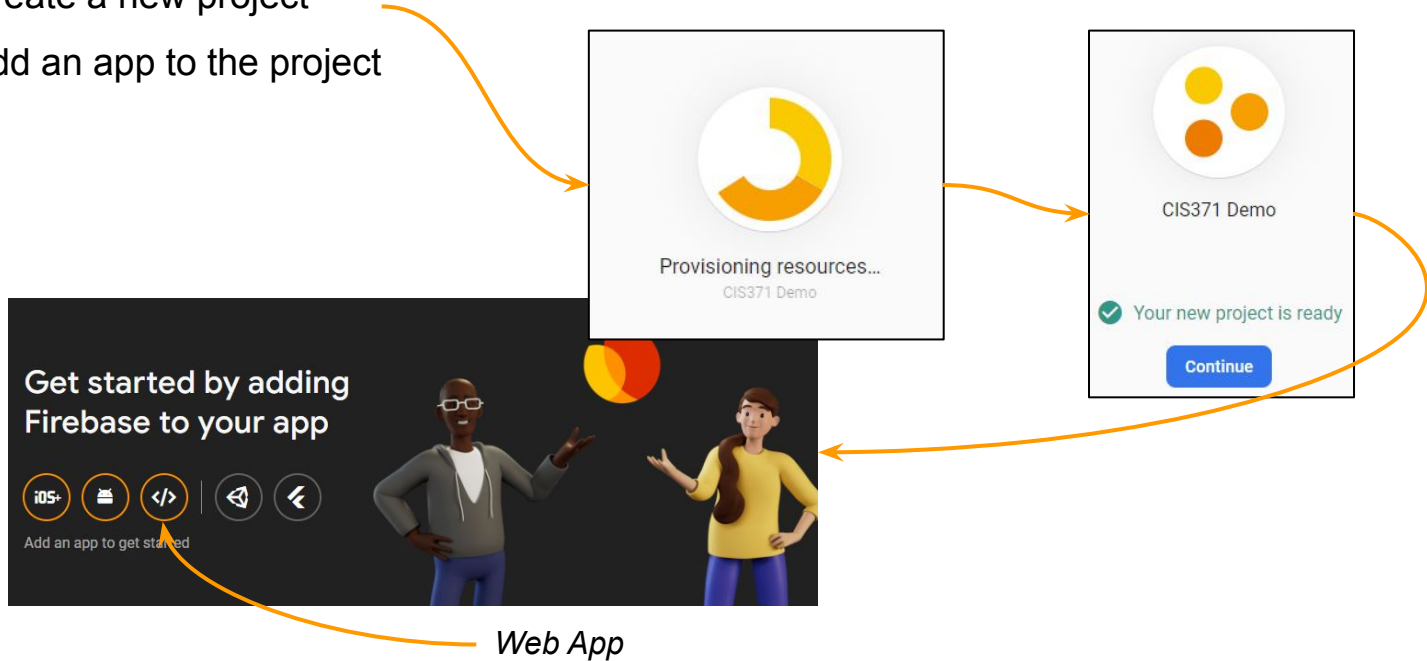
	Realtime DB	Cloud Firestore
Auto-generated Key	Time-based	Not time-based
Write operations	Max 1000 writes/second	Max 10,000 writes/second
Offline support	iOS and Android clients	iOS, Android, and Web clients
Concurrent Connections	Max 200,000	Max 1,000,000
Data Model	Giant JSON tree	Hierarchy of Collections ("Tables") and Documents ("Records")
Queries	Deep (slower performance), fetching a node will return the entire subtree of the node	Shallow (better performance), it is possible to fetch a document without its "children"
	Queries can use sorting or filtering (but not both)	Queries can use sorting and filtering

Local Storage, Local Events, & Global Events



Creating a new WebApp

1. Use a Google account to login to [Firebase Console](#)
2. Create a new project
3. Add an app to the project



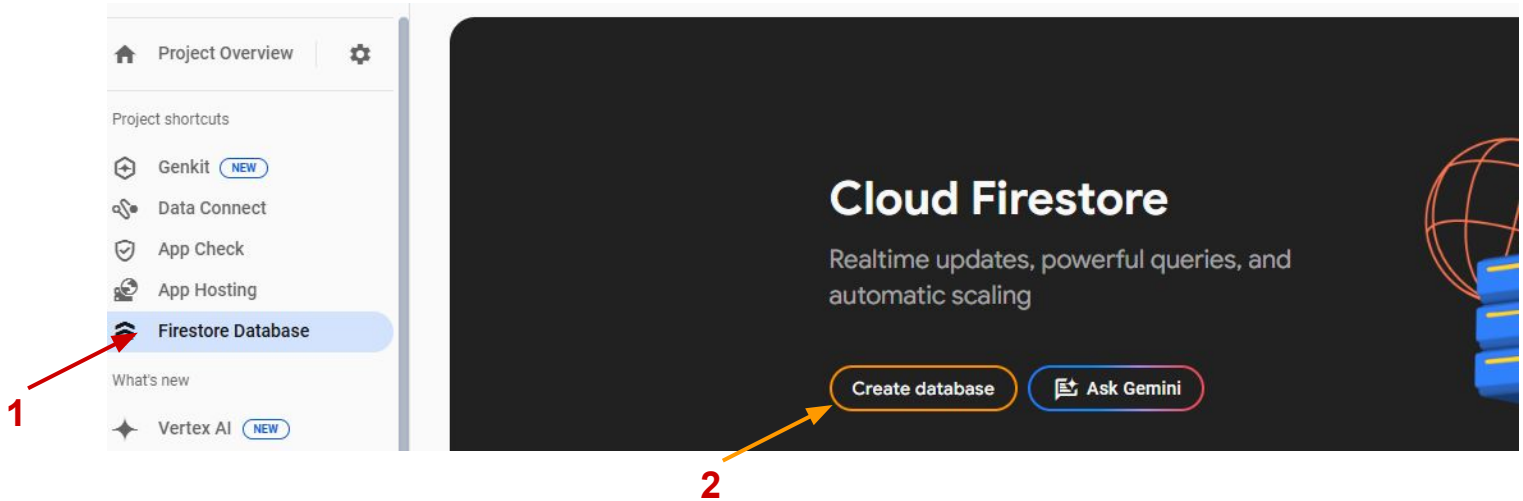
Creating a new WebApp

The screenshot shows the Firebase console interface for a project named 'CIS371 Demo'. The left sidebar contains navigation options: Project Overview, Product categories, Build, Release & Monitor, Analytics, Engage, and All products. A red arrow labeled '1' points to the 'Build' option. The main content area shows the 'Project settings' dropdown menu with options: Project settings, Users and permissions, and Usage and billing. The 'Web apps' section lists a single web app named 'CIS371Demo'. The 'SDK setup and configuration' section shows three radio buttons: npm, CDN, and Config. A red arrow labeled '2' points to the 'Config' option. Below this, there is a text input for 'App nickname' (CIS371Demo) and 'App ID' (1:988644841148:web:3b246d80be5da0bccb62b9). A 'Link to a Firebase Hosting site' button is also present. A red box highlights the Firebase configuration object code snippet:

```
// For Firebase JS SDK v7.20.0 and later, measurementId is optional
const firebaseConfig = {
  apiKey: "cis371-demo-770dc.firebaseio.com",
  authDomain: "cis371-demo-770dc.firebaseio.com",
  projectId: "cis371-demo-770dc.firebaseio.com",
  storageBucket: "cis371-demo-770dc.firebaseio.com",
  messagingSenderId: "988644841148",
  appId: "1:988644841148:web:3b246d80be5da0bccb62b9",
  measurementId: "G-X2KWMBZYFB"
};
```

Below the code snippet, there is a note: 'Are you using npm and a bundler like webpack or Rollup? Check out the [modular SDK](#).' and a link to 'Learn more about Firebase for web: [Get Started](#), [Web SDK API Reference](#), [Samples](#)'.

Initialize Firestore



Local Project Setup (On Your Computer)

Project Setup & Initialization

```
yarn init -y  
yarn add firebase
```

OR

```
npm init -y  
npm install firebase
```

```
import { initializeApp, FirebaseApp } from "firebase/app";  
import { getFirestore, Firestore } from "firebase/firestore";  
  
const firebaseConfig = {  
  apiKey: "your-api-key-goes-here",  
  authDomain: "your-project-name-here.firebaseio.com",  
  databaseURL: "https://your-project-name-here.firebaseio.com",  
  projectId: "your-project-name-here",  
  storageBucket: "your-project-name.appspot.com",  
  messagingSenderId: "xxxxxxx"  
};  
  
// Initialize Firebase  
const myapp: FirebaseApp = initializeApp(firebaseConfig);  
const db: Firestore = getFirestore(myapp);
```

// COPY this from your Firebase Console

Database Dashboard

- Browse and Modify Data
- Security Rules (default settings: user authentication required)

```
// Allow read/write access on all documents to any user signed in to the application
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if request.auth != null;
    }
  }
}
```

[Cloud Firestore Security Rules](#)

Data Model: Hierarchy of Collections-Documents

- Hierarchical structure
 - The “root” holds one or more collections
 - A collection consists of one or more documents
 - A document is one or more key-value pairs
 - A value in a document may refer to a subcollection (1-to-many relationships)
- Data Types in a document
 - string, number, boolean, array, timestamp, map (kv-pairs), geolocation
 - Reference to a subcollection

SQL	Cloud Firestore
Tables	Collections
Rows	Documents
Primary Key	Document ID
Fields	key-value pairs

Data Model: Hierarchy of Collections-Documents

State (SQL table)

Abbrev (PK)	Name	Capital
AK	Alaska	Juneau
AL	Alabama	Montgomery
FL	Florida	Tallahassee

NatlPark (SQL table)

Code (PK)	Name	Location
U6123	Arches	Utah
C1632	Black Canyon	Colorado

SQL Table $\Rightarrow$ Firestore Collection
SQL Primary Key $\Rightarrow$ Firestore Document ID/"name"
SQL Table Row $\Rightarrow$ Firestore Document

States (Collection of 3 Documents)

AK

Name: Alaska
Capital: Juneau

AL

Name: Alabama
Capital: Montgomery

FL

Name: Florida
Capital: Tallahassee

Parks (Collection of 2 Documents)

U6123

Name: Arches
Location: Utah

C1632

Name: Black Canyon
Location: Colorado

All Firestore Collection/Doc Manipulation Functions return a Promise

Firestore Functions (version 9.x)

- Functions for creating references
 - `collection(refToFirestore, "path/to/collection")`
 - `doc(refToFirestoreOrCollection, "path/to/your/document")`
 - `query(refToCollection, _____)`
- **Retrieval functions**
 - `getDoc(refToDoc)`
 - `getDocs(refToCollection)`
- **Manipulation functions**
 - `addDoc(refToColl, { new_content_object })`
 - `setDoc(refToDoc, { new_content_object })`
 - `updateDoc(refToDoc, { new_content_object })`
 - `deleteDoc(refToDoc)`
- Update listener on `Snapshot ( )` **(specific to Firebase)**

C
R
U
D

CRUD Operations (Summary)

	Collection	Document
Create	Implied when a doc is created	<pre>// Option #1 const collPar = collection(db, "cName"); addDoc(collPar, { /* new content here */ }); // Option #2 const myDoc = doc(db, "cName", "docName"); setDoc(myDoc, { /* new content here */ })</pre>
Read	<pre>const myC = collection(db, "cName"); getDocs(myC).then(____);</pre>	<pre>const myDoc = doc(____, _____, ____); getDoc(myDoc).then(____);</pre>
Update	N/A	<pre>const myDoc = doc(____, _____, ____); updateDoc(myDoc, { /* content */ }).then(____);</pre>
Delete	N/A	<pre>const myDoc = doc(____, _____, ____); deleteDoc(myDoc).then(____);</pre>

CRUD Operations: Create Doc (own Doc ID)

```
// Use "AK" as the primary key for the tuple  
INSERT INTO states (abbrev, name, capital) VALUES("AK", "Alaska", "Juneau")
```

SQL

states (SQL table)

Abbrev (PK)	Name	Capital
AK	Alaska	Juneau
AL	Alabama	Montgomery
FL	Florida	Tallahassee

```
import { DocumentReference, setDoc, doc } from "firebase/firestore";  
// Option #1: Use file name syntax for doc path  
// Primary key "AK" becomes doc id  
const doc1: DocumentReference = doc(db, "states", "AK");  
setDoc(doc1, { name: "Alaska", capital: "Juneau" })  
  .then(() => {  
    console.log("New doc added");  
  })  
  .catch((err: any) => {  
    /* your code here */  
  });
```

Firestore in TS

CRUD Operations: Create Doc (automatic Doc ID)

```
INSERT INTO states (name, capital) VALUES("Alaska", "Juneau")
```

SQL

states (SQL table)

Name	Capital
Alaska	Juneau
Alabama	Montgomery
Florida	Tallahassee

```
import { CollectionReference, addDoc, doc } from "firebase/firestore";

const myColl: CollectionReference = collection(db, "states");
addDoc(myColl, { name: "Alaska", capital: "Juneau" })
  .then(() => {
    console.log("New doc added");
  })
  .catch((err: any) => {
    /* your code here */
  });
```

Firestore in TS

CRUD Operations: Create Docs from Array

Firestore in TS

```
import {
  DocumentReference,
  setDoc,
  doc,
  collection,
  addDoc,
} from "firebase/firestore";

const stateArr = [
  { abbrev: "CA", name: "California", capital: "Sacramento" },
  { abbrev: "CO", name: "Colorado", capital: "Denver" },
  // more data here
];

// Option 1: Use state abbreviation as document ID
stateArr.forEach(async (st: any) => {
  const stateDoc = doc(db, "states", st.abbrev); // Use Abbreviation as document ID
  await setDoc(stateDoc, { name: st.name, capital: st.capital });
});

// Option 2: Let Firestore generate automatic
const myStateColl = collection(db, "states"); // Do this outside .forEach
stateArr.forEach(async (st: any) => {
  await addDoc(myStateColl, { name: st.name, capital: st.capital });
});
```

await vs. .then()

CRUD Operations: Read All Documents

```
SELECT * FROM states
```

SQL

states (SQL table)

Abbrev (PK)	Name	Capital
AK	Alaska	Juneau
AL	Alabama	Montgomery
FL	Florida	Tallahassee

```
// Assume saved data has the
// following structure
type StateType = {
  abbrev: string;
  name: string;
  capital: string;
};
```

```
import {
  CollectionReference,
  collection,
  QuerySnapshot,
  QueryDocumentSnapshot,
  getDocs,
} from "firebase/firestore";

const myStateColl: CollectionReference = collection(db, "states");

getDocs(myStateColl).then((qs: QuerySnapshot) => {
  qs.forEach((qd: QueryDocumentSnapshot) => {
    const stateData = qd.data() as StateType;
    const docId = qd.id; // Fixed 'cost' to 'const'
    // More code here to manipulate stateData
  });
});
```

Firestore in TS

CRUD Operations: Read A Specific Document

```
// Select a tuple with a known primary key  
SELECT * FROM states WHERE abbrev = "FL"
```

SQL

states (SQL table)

Abbrev (PK)	Name	Capital
AK	Alaska	Juneau
AL	Alabama	Montgomery
FL	Florida	Tallahassee

```
// Assume saved data has the  
// following structure  
type StateType = {  
  abbrev: string;  
  name: string;  
  capital: string;  
};
```

```
import {  
  DocumentReference,  
  doc,  
  DocumentSnapshot,  
  getDoc,  
} from "firebase/firestore";  
// FL is a document ID  
const myDoc: DocumentReference = doc(db, "states/FL");  
getDoc(myDoc).then((qd: DocumentSnapshot) => {  
  if (qd.exists()) {  
    const stateData = qd.data() as StateType;  
    // More code here to manipulate stateData  
  }  
});
```

Firestore in TS

CRUD Operations: Fetch Document(s) Where...

// Select tuples satisfying some conditions
SELECT * FROM states WHERE name = "Florida"

SQL

states (SQL table)

Abbrev (PK)	Name	Capital
AK	Alaska	Juneau
AL	Alabama	Montgomery
FL	Florida	Tallahassee

```
// Assume saved data has the  
// following structure  
type StateType = {  
  abbrev: string;  
  name: string;  
  capital: string;  
};
```

```
import {  
  Query,  
  getDocs,  
  collection,  
  where,  
  query,  
  QuerySnapshot,  
  QueryDocumentSnapshot,  
} from "firebase/firestore";  
const getFL: Query = query(  
  collection(db, "states"),  
  where("name", "==", "Florida")  
);  
getDocs(getFL).then((qs: QuerySnapshot) => {  
  qs.forEach((qd: QueryDocumentSnapshot) => {  
    const stateData = qd.data() as StateType;  
    // More code here to manipulate stateData  
  });  
});
```

Firestore in TS

CRUD Operations: Fetch Document(s) Where...

// Select tuples satisfying some conditions

```
SELECT * FROM states WHERE population > 10_000_000
```

SQL

states (SQL table)

Name	Capital	Population
California	Sacramento	39_123_612
Michigan	Lansing	8_432_911
Florida	Tallahassee	26_222_943

```
// Assume saved data has the
// following structure
type StateType = {
  name: string;
  capital: string;
  population: number;
};
```

```
import {
  Query,
  getDocs,
  collection,
  query,
  where,
  QuerySnapshot,
  QueryDocumentSnapshot,
} from "firebase/firestore";
const aboveTenMil: Query = query(
  collection(db, "states"),
  where("population", ">", 10_000_000)
);
getDocs(aboveTenMil).then((qs: QuerySnapshot) => {
  qs.forEach((qd: QueryDocumentSnapshot) => {
    const stateData = qd.data() as StateType;
    // More code here to manipulate stateData
  });
});
```

Firestore in TS

CRUD Operations: Fetch Document(s) Where...

SQL

```
// Select tuples satisfying some conditions
SELECT * FROM states WHERE population > 10_000_000
AND population < 15_000_000
```

states (SQL table)

Name	Capital	Population
California	Sacramento	39_123_612
Michigan	Lansing	8_432_911
Florida	Tallahassee	26_222_943

```
// Assume saved data has the
// following structure
type StateType = {
  name: string;
  capital: string;
  population: number;
};
```

```
import {
  Query,
  getDocs,
  collection,
  query,
  where,
  QuerySnapshot,
  QueryDocumentSnapshot,
} from "firebase/firestore";

const aboveTenMil: Query = query(
  collection(db, "states"),
  where("population", ">", 10_000_000),
  where("population", "<", 15_000_000)
);

getDocs(aboveTenMil).then((qs: QuerySnapshot) => {
  qs.forEach((qd: QueryDocumentSnapshot) => {
    const stateData = qd.data() as StateType;
    // More code here to manipulate stateData
  });
});
```

Firestore in TS

Available Query Where Operators

Operator	Example	SQL Equivalent
<, <=, ==, >=, >	<code>where("population", ">", 20_000_000)</code>	<code>WHERE population > 20000000</code>
!=	<code>where("name", "!= ", "Andy")</code>	<code>WHERE name != "Andy"</code>
in	<code>where("city", "in", ["Ada", "Flint"])</code>	<code>WHERE city == "Ada" OR city == "Flint"</code>
not-in	<code>where("city", "not-in", ["Ada", "Flint"])</code>	<code>WHERE city != "Ada" AND city != "Flint"</code>

Operator	Example (courses must be an ARRAY)
array-contains	<code>// Has this student taken MTH200?</code> <code>where("courses", "array-contains", "MTH200")</code>
array-contains-any	<code>// Has this student taken either MTH200 or</code> <code>STA215? where("courses", "array-contains-any",</code> <code>["MTH200", "STA215"])</code>

Query Limitations

```
// Multiple .where() on the same field
const q = query(
  collection(__, "states"),
  where("population", ">=", 5_000_000),
  where("population", "<=", 10_000_000)
);
getDocs(q).then(() => {
  /* code */
});
```

OK

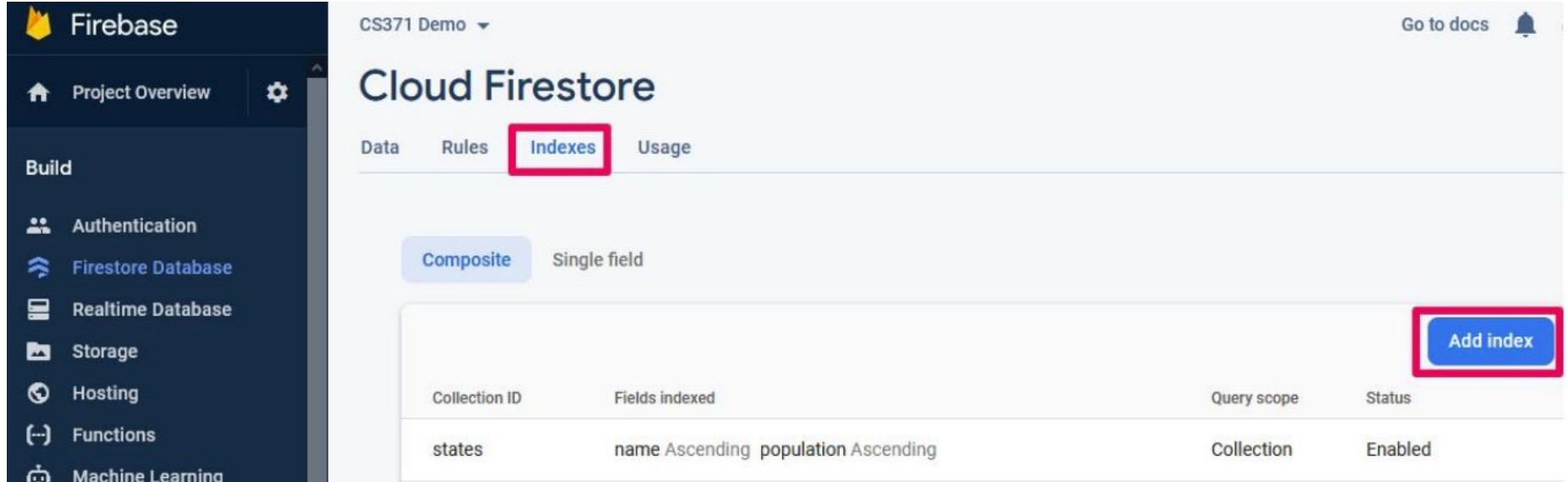
```
// Can't use inequalities on two different fields
query(
  collection(__, "states"),
  where("population", ">=", 5_000_000),
  where("area", "<=", 200_000)
);
```

Not OK

```
// Multiple .where on different fields
// require a composite index on both fields
// At most one inequality comparison!!
const q = query(
  collection(__, "students"),
  where("major", "==", "MATH"),
  where("gpa", ">=", 3.0)
);
getDocs(q).then(/* more code */);
```

OK

Building Composite Index



CS371 Demo

Go to docs

Cloud Firestore

Data Rules **Indexes** Usage

Composite Single field

Add index

Collection ID	Fields indexed	Query scope	Status
states	name Ascending population Ascending	Collection	Enabled

Order of index build does matter!!!

Exercises: Try Your First App with Firestore

1: Create a Firebase Project

- Go to Firebase Console, click "**Create a Firebase project**"
- Name your project **CIS371**, disable Google Analytics (optional), then click **Continue**

2. Register Your App

- In the **CIS371** project dashboard, click "</>" to add a **Web App**
- Name your app **CustomDrinkMaker**
- Click **Register App**
- Follow the instructions to **add Firebase SDK** to your web app (copy the config snippet)

3: Set Up Firestore

- In the left sidebar, go to "**Build**" → **Firestore Database**
- Click **Create Database**
- Choose your Firestore location (United States)
- Start in **Test Mode** (for development)

Ready to Code!

- *Initialize Firebase in your vue3 app using the provided config*
- *Test your connection and start reading/writing data from Firestore*